Development of a Software Engineering Chatbot: A Comparative Study of From-Scratch, Fine-Tuning, and Retrieval-Augmented Generation Approaches

HAITI Aya, ELBALLAOUI Hasnae, NECHDA Salma, MAZGOUT Rim
National School of Applied Sciences of Al Hoceima
Al Hoceima, Morocco

Pr. Mohammed CHERRADI (Supervisor)
National School of Applied Sciences of Al Hoceima
Al Hoceima, Morocco
m.cherradi@uae.ac.ma

***Abstract*—Conversational agents have become a central component of modern educational and technical assistance systems, particularly in specialized domains such as software engineering. However, building a chatbot capable of producing accurate, contextually grounded, and pedagogically useful answers remains a non-trivial task, mainly due to the risk of hallucination and the difficulty of covering a broad technical vocabulary. In this work, we present the development of a chatbot specialized in software engineering and conduct a comparative study of three deep-learning-based approaches: a from-scratch Sequence-to-Sequence model with attention, a Fine-Tuned FLAN-T5 model, and a Retrieval-Augmented Generation (RAG) pipeline combining FAISS, BM25, a CrossEncoder reranker, and a LoRA-adapted FLAN-T5 generator. Each approach was implemented, trained, and evaluated on dedicated software-engineering corpora using standard metrics such as BLEU, Accuracy@K, MRR, ROUGE-1, ROUGE-L, and semantic similarity. The from-scratch model reached a BLEU score of only 0.0003, confirming the difficulty of training a generative chatbot from zero with a moderately sized corpus. The fine-tuning approach significantly improved response quality on known questions but remained tightly coupled to the training distribution. The RAG-based system achieved a ROUGE-L of 0.6696, a ROUGE-1 of 0.6864, and a semantic similarity of 0.9449, while substantially reducing hallucinations by grounding answers in retrieved knowledge. Based on these results, the RAG architecture was selected as the most suitable solution. The main contribution of this work is a systematic, end-to-end comparison of the three paradigms within the same specialized domain, providing practical guidance for the design of academic conversational assistants.**

***Index Terms*—Software Engineering, Chatbot, Deep Learning, Natural Language Processing, Fine-Tuning, Retrieval-Augmented Generation, FLAN-T5, FAISS, BM25, CrossEncoder, Semantic Search, Seq2Seq, Attention Mechanism, Gradio.**

I. Introduction

Artificial intelligence and, in particular, deep learning have transformed the way humans interact with digital systems. Among the most visible products of this transformation are conversational agents, commonly known as chatbots, which allow users to communicate with software in natural language. Modern chatbots are no longer limited to scripted rule-based dialogues; thanks to large pre-trained language models, they can now handle nuanced and complex queries, support multiple languages, and adapt to a wide variety of domains [1]–[3].

In educational and technical contexts, chatbots are increasingly used as on-demand tutors that complement traditional teaching resources. Software engineering, in particular, is a discipline where students frequently need precise explanations about concepts such as object-oriented programming, UML modeling, design patterns, software testing, databases, APIs, version control, DevOps, and agile methodologies. The diversity of available documentation, forums, and online courses can be overwhelming, and the answers found in these sources are often inconsistent. A chatbot specialized in software engineering can therefore play an important pedagogical role by providing concise, structured, and reliable explanations tailored to learners' questions.

Designing such a specialized chatbot, however, raises several challenges. A general-purpose chatbot may answer common questions but often fails when domain-specific accuracy is required. Building a generative chatbot from scratch typically demands very large training corpora and significant computational resources. Fine-tuning a pre-trained model improves quality but remains dependent on the coverage of the training data and is still susceptible to hallucinations. Retrieval-Augmented Generation (RAG) attempts to address these issues by grounding the generation in retrieved knowledge, but its effectiveness depends on the quality of the underlying knowledge base and retrieval pipeline [4].

Most existing works on conversational agents and question answering focus on a single approach evaluated in isolation. There is comparatively little work that implements and compares from-scratch generation, domain-specific fine-tuning, and retrieval-augmented generation within the same

specialized domain. The present work addresses this gap by developing three distinct chatbots for software engineering and systematically comparing them along technical, quantitative, and qualitative dimensions.

The contributions of this paper are threefold. First, we implement a from-scratch Sequence-to-Sequence chatbot with attention, trained on a corpus collected from Stack Overflow, CodeSearchNet [5], and GitHub Issues, in order to study the intrinsic difficulty of generative training in this domain. Second, we fine-tune the FLAN-T5 model on a curated software-engineering question-answering dataset, coupled with a FAISS-based semantic retrieval module and a CrossEncoder reranker. Third, we design and evaluate a complete RAG pipeline that combines multilingual E5 embeddings, BM25 lexical retrieval, hybrid score combination, CrossEncoder reranking, prompt construction, and contextual generation by a FLAN-T5 model adapted with LoRA. The three systems are integrated into Gradio-based interfaces and evaluated using BLEU, Accuracy@K, MRR, ROUGE-1, ROUGE-L, and semantic similarity.

The rest of this paper is structured as follows. Section II reviews the related literature on chatbots, NLP, sequence-to-sequence models, pre-trained language models, fine-tuning, retrieval-augmented generation, and hybrid retrieval. Section III describes the methodology, including the datasets, preprocessing, model architectures, training procedures, evaluation metrics, and rationale for method selection. Section IV reports and discusses the experimental results obtained with the three approaches and presents a comparative analysis. Section V outlines the main limitations of our work and proposes directions for future research. Section VI concludes the paper.

II. Literature Review

A. Chatbots and Conversational Agents

Chatbots have evolved through three main generations. Early rule-based systems, such as ELIZA, relied on handcrafted patterns and decision trees, which limited them to scripted interactions. Statistical chatbots later introduced probabilistic intent classifiers and Hidden Markov Models, improving robustness but still constrained by feature engineering. The most recent generation is based on deep learning and, in particular, Transformer architectures [1], which enable open-domain, context-aware, and fluent dialogues.

B. NLP for Question Answering

Question answering (QA) is one of the canonical tasks of natural language processing. It combines several sub-tasks such as text cleaning, tokenization, language detection, semantic understanding, retrieval, and generation. In a specialized domain like software engineering, QA systems must handle a mixture of natural language and technical terminology (e.g., `git rebase`, REST, UML, Singleton), which makes the task more challenging than open-domain QA.

C. Sequence-to-Sequence Models and Attention

Sequence-to-Sequence (Seq2Seq) models [6] were introduced for machine translation and quickly became a standard tool for sequence transduction tasks, including dialogue generation. A Seq2Seq model is composed of an encoder that maps the input sequence into a hidden representation and a decoder that generates the output sequence word by word. The fixed-size bottleneck of the basic architecture was alleviated by the attention mechanism proposed by Bahdanau et al. [7], which allows the decoder to softly align with relevant parts of the input at every decoding step.

D. Pre-trained Language Models and Fine-Tuning

Pre-trained language models such as BERT [8], GPT, T5 [2], and FLAN-T5 [3] have set a new standard in NLP. They learn rich linguistic representations from large corpora and can subsequently be adapted to downstream tasks via fine-tuning. Full fine-tuning updates all parameters and is computationally expensive, while Parameter-Efficient Fine-Tuning (PEFT) techniques such as LoRA [9] freeze most parameters and add a small number of trainable low-rank matrices, drastically reducing memory and training time. FLAN-T5 is particularly suited to question-answering applications because it has been instruction-tuned on a wide range of tasks.

E. Retrieval-Augmented Generation

Retrieval-Augmented Generation [4] combines a retriever and a generator: relevant documents are first retrieved from a knowledge base and then provided as context to the generator. This paradigm is widely used for knowledge-intensive tasks because it reduces hallucinations, improves factual accuracy, and allows the knowledge base to be updated independently of the model.

F. Semantic Search and Hybrid Retrieval

Semantic search relies on dense embeddings produced by models such as Sentence-BERT [10] or multilingual E5 [11], and on vector indexes such as FAISS [12] for efficient nearest-neighbor search. Lexical retrieval methods such as BM25 [13] remain effective for technical vocabulary that requires exact term matching. Hybrid retrieval, which combines dense and sparse signals, often improves robustness in specialized domains. Reranking with a CrossEncoder is a common additional step that re-evaluates the top candidates with finer-grained query–document interaction.

G. Gap Addressed by Our Work

Although chatbots, fine-tuning, and RAG have all been extensively studied, most existing works focus on a single approach evaluated on a generic benchmark. Cross-approach comparisons in a specialized domain such as software engineering remain rare. Our work fills this gap by implementing and comparing three approaches (from-scratch, fine-tuning, RAG) for the same task and the same domain, using consistent evaluation protocols.

TABLE I
DATASETS USED IN EACH APPROACH.

Approach	Data sources	Total	Train	Val / Test
From-Scratch	Stack Overflow, CodeSearchNet, GitHub Issues	12,693	10,154	1,269 val + 1,270 test
Fine-Tuning	Curated SE QA dataset	655	589	66 val
RAG	Augmented SE QA corpus	809	728 (725 filt.)	81 val

III. METHODOLOGY

To investigate which approach is best suited for a software-engineering chatbot, we designed three distinct pipelines based on the same global problem formulation: given a user question q related to software engineering, generate a clear, correct, and pedagogical answer $\hat{a}$. The three approaches differ in how they collect, represent, and exploit knowledge. The experimental work was carried out on Kaggle using a Tesla T4 GPU and the standard Python stack (TensorFlow, Keras, PyTorch, Hugging Face Transformers, SentenceTransformers, FAISS, rank-bm25, peft, evaluate, and Gradio).

A. Dataset

Each approach relies on a dedicated dataset adapted to its specific requirements. The from-scratch approach uses a large raw corpus collected from Stack Overflow, CodeSearchNet [5], and GitHub Issues, in order to maximize the volume of training examples. The fine-tuning approach uses a smaller but curated and balanced corpus covering software-engineering categories such as architecture, OOP, design patterns, testing, DevOps, databases, UML, agile, APIs, and Git. The RAG approach extends the fine-tuning corpus through data augmentation (paraphrasing, conversational reformulation) to increase robustness to question variability. Table I summarizes the datasets used in each approach.

The from-scratch corpus aims to train a generative model from zero, the fine-tuning corpus is used to adapt FLAN-T5 to the target domain, and the RAG corpus serves both as the knowledge base for retrieval and as the training set for the LoRA-adapted generator.

B. Data Preprocessing

Several preprocessing steps were applied across the three approaches. Raw texts were cleaned by removing HTML tags, unsupported characters, and overly long code blocks. Duplicates and empty entries were removed. Questions and answers were normalized (lowercasing where appropriate, whitespace cleaning, punctuation handling). A quality filter discarded malformed entries containing fragments of code or docstring patterns such as `:param`, `:return`, or `def`.

For the from-scratch approach, additional steps included: vocabulary construction (final size: 13,445 tokens), sequence padding, addition of special tokens `<PAD>`, `<BOS>`, `<EOS>`, `<UNK>`, and construction of an embedding matrix combining GloVe [14] (300 dimensions) and Word2Vec [15] domain-specific embeddings.

For the fine-tuning and RAG approaches, the tokenizers shipped with the pre-trained models were used directly. Sentence embeddings were generated using `sentence-transformers/paraphrase-multilingual-Mini` (384 dimensions) for the fine-tuning approach and `intfloat/multilingual-e5-large` (1024 dimensions) for the RAG approach. The E5 convention of prefixing queries with `"query:"` and passages with `"passage:"` was respected to maximize retrieval accuracy. FAISS indexes were then constructed (655 vectors for fine-tuning, 809 vectors for RAG) to enable fast similarity search.

C. Models

1) From-Scratch Seq2Seq with Attention: The first model is a Sequence-to-Sequence architecture trained from scratch in TensorFlow/Keras. The encoder is a Bidirectional LSTM that processes the tokenized question and produces a sequence of hidden states capturing both left and right context. A Bahdanau attention layer [7] computes, at each decoding step t, a context vector

$$c_t = \sum_{i=1}^{T_x} \alpha_{t,i}\, h_i, \qquad \alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{k=1}^{T_x} \exp(e_{t,k})}, \tag{1}$$

where $e_{t,i} = v^\top \tanh(W_1 h_i + W_2 s_{t-1})$ measures the relevance between the previous decoder state s_{t-1} and the encoder hidden state h_i. The decoder is an LSTM with dropout 0.3 that generates the answer word by word from the context vector and the previously generated tokens. This approach offers full control over the architecture but relies entirely on the training data with no external knowledge.

2) Fine-Tuned FLAN-T5 Model: The second model is based on `google/flan-t5-base`, a pre-trained instruction-tuned Transformer with approximately 250M parameters. The model is fine-tuned on the curated software-engineering corpus to learn the specific terminology and answer style of the domain. At inference time, a FAISS semantic retrieval module first selects candidate question–answer pairs close to the user query; a CrossEncoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) then reranks the candidates, and the fine-tuned FLAN-T5 generates the final response. A language-detection and translation module (English/French) supports bilingual usage. This approach leverages strong pre-trained linguistic knowledge but still relies primarily on what the model has memorized.

3) RAG-Based Chatbot: The third model implements a complete RAG pipeline. The user question is first cleaned and translated if necessary. Hybrid retrieval is then performed: a dense FAISS search using `multilingual-e5-large` embeddings and a sparse BM25 search are combined into a hybrid score

$$s_{\text{hybrid}}(q,d) = \lambda\, \tilde{s}_{\text{dense}}(q,d) + (1-\lambda)\, \tilde{s}_{\text{BM25}}(q,d), \tag{2}$$

TABLE II
EVALUATION METRICS USED IN THE PROJECT.

Metric	Used for	Meaning
BLEU	From-Scratch	Lexical similarity
Accuracy@K	FT, RAG (retr.)	Correct doc. in top-K
MRR	FT, RAG (retr.)	Mean reciprocal rank
ROUGE-1	Generation	Unigram overlap
ROUGE-L	Generation	Longest common subseq.
Sem. Similarity	Generation	Cosine of embeddings

TABLE III
RETRIEVAL RESULTS FOR THE FINE-TUNING APPROACH.

Metric	Value
Accuracy@1	1.0000
Accuracy@3	1.0000
Accuracy@5	1.0000
MRR	1.0000
Number of evaluation examples	66

with $\lambda = 0.6$ giving slightly more weight to semantic similarity. The top candidates are reranked by a CrossEncoder, the best contexts are integrated into a prompt template that instructs the model to explain rather than copy, and a FLAN-T5 model adapted with LoRA generates the final answer. LoRA [9] adapters are applied to the q and v projection matrices of every attention block, with rank $r = 16$, scaling $\alpha = 32$, and dropout 0.05. Only $1{,}769{,}472$ of the $249{,}347{,}328$ model parameters (0.7096%) are trainable. The system supports both English and French and is deployed via Gradio. The key advantage is that the generator is grounded in retrieved knowledge, which strongly reduces hallucinations.

D. Training

For the from-scratch approach, training was performed in TensorFlow/Keras on the Tesla T4 GPU using the Adam optimizer with an initial learning rate of 10^{-3}, sparse categorical cross-entropy with padding masking, and teacher forcing. Early stopping based on validation loss was applied to mitigate overfitting. Despite these precautions, training remained slow and unstable.

For the fine-tuning approach, FLAN-T5-base was fine-tuned for 3 epochs with a batch size of 8, a maximum input length of 256 tokens, a maximum target length of 192 tokens, a learning rate of 1×10^{-4}, mixed-precision (FP16) training, and early stopping with patience 2. The complete fine-tuning lasted approximately 1.3 minutes on the Tesla T4 GPU.

For the RAG approach, FLAN-T5-base was fine-tuned with LoRA on the augmented corpus. The trainable parameters (0.7096% of the full model) make training significantly cheaper than full fine-tuning. The retrieval and reranking modules are not trained but instantiated from publicly available pre-trained models. The complete inference pipeline runs in real time within the Gradio interface.

E. Evaluation Metrics

Different approaches were evaluated with metrics adapted to their nature. The from-scratch generator was evaluated with BLEU, while the fine-tuning and RAG approaches were evaluated with retrieval metrics (Accuracy@K, MRR) and generation metrics (ROUGE-1, ROUGE-L, semantic similarity). Table II summarizes these metrics.

F. Rationale for Method Selection

The choice of these three approaches reflects a deliberate methodological progression. The from-scratch Seq2Seq baseline was selected to study the intrinsic difficulty of building a generative chatbot from zero and to understand the role of attention in encoder–decoder architectures. The fine-tuning approach was chosen to evaluate the benefit of leveraging a strong pre-trained model (FLAN-T5) and adapting it to the target domain at a moderate cost. The RAG approach was chosen specifically to address the main weakness of the fine-tuning-only solution: the risk of hallucination on questions that are not well represented in the training data. The combination of FAISS (dense retrieval), BM25 (sparse retrieval), and a CrossEncoder (reranking) was selected because it captures both semantic similarity and exact keyword matching, which is essential in a domain where terms such as REST, UML, or Git carry very specific meaning.

IV. RESULTS AND DISCUSSION

This section reports and discusses the experimental results obtained with the three approaches. The metrics used were defined in Section III-E.

A. From-Scratch Results

The from-scratch Seq2Seq model with attention obtained a BLEU score of 0.0003 on the test set. Qualitative inspection of the generated answers confirmed that the model failed to learn a reliable mapping between software-engineering questions and answers: most outputs consisted of incoherent sequences of technical words rather than meaningful explanations. Although the model trained successfully from a technical standpoint and demonstrated the inner workings of an encoder–attention–decoder architecture, the corpus of approximately 12,693 examples proved insufficient to learn the complex distribution of software-engineering language. The from-scratch approach therefore served as a valuable pedagogical baseline but could not be retained as a production-ready solution.

B. Fine-Tuning Results

The fine-tuning approach showed a substantial improvement over the from-scratch baseline. Both the retrieval and the generation components reached very high scores on the validation set, as reported in Tables III and IV.

These perfect scores must be interpreted with caution. They reflect very strong performance on validation questions that are close to the training distribution and to the indexed knowledge base, but they do not guarantee robustness on out-of-domain or strongly reformulated questions. In fact, identical scores of 1.0000 across both retrieval and generation metrics strongly

TABLE IV
GENERATION RESULTS FOR THE FINE-TUNING APPROACH.

Metric	Value
ROUGE-L	1.0000
ROUGE-1	1.0000
Semantic Similarity	1.0000

TABLE V
GENERATION RESULTS FOR THE RAG APPROACH.

Metric	Score
ROUGE-L	0.6696
ROUGE-1	0.6864
Semantic Similarity	0.9449

suggest a substantial overlap between the validation set and the indexed/training data, i.e. a form of evaluation leakage rather than genuine generalization. In practice, the fine-tuned model was observed to produce confident but occasionally hallucinated answers when faced with questions outside the curated corpus, which motivated the move to a RAG-based architecture.

C. RAG Results

The RAG approach was evaluated on 81 validation examples. The retrieval components (FAISS + BM25 + CrossEncoder) achieved perfect Accuracy@1, Accuracy@3, Accuracy@5, and MRR. The generation results are reported in Table V.

The lower ROUGE scores compared to the fine-tuning-only approach reflect the fact that the RAG generator does not simply copy reference answers but reformulates them in a more natural and explanatory style. This is supported by the very high semantic similarity of 0.9449, which indicates that the generated answers preserve the meaning of the reference answers even when they differ at the surface level. In other words, RAG sacrifices a small amount of lexical overlap in exchange for more natural, grounded, and pedagogical responses. Combined with hybrid retrieval and reranking, this design makes the RAG system more robust to question variability and significantly less prone to hallucination.

D. Comparative Discussion

Table VI provides a synthetic comparison of the three approaches along eight criteria relevant to the design of a specialized chatbot.

The from-scratch approach is conceptually simple but ineffective at this dataset size, illustrating the limits of training generative models without large-scale pre-training. The fine-tuning approach is significantly more effective and produces well-formed answers, but its excellent metrics on the validation set hide a real risk of hallucination on unseen questions. The RAG approach offers the most balanced trade-off: it inherits the linguistic competence of the pre-trained FLAN-T5 model, benefits from a curated knowledge base accessed through hybrid retrieval, and uses reranking to ensure that the most relevant context is passed to the generator.

TABLE VI
COMPARATIVE STUDY OF THE THREE APPROACHES.

Criterion	From-Scratch	Fine-Tuning	RAG
Training strategy	Seq2Seq + attention from zero	FLAN-T5 fine-tuned on SE corpus	FLAN-T5 + LoRA + hybrid retrieval
Data requirement	Very large	Moderate	Moderate + KB
External knowledge	None	Limited (FAISS)	Strong (FAISS + BM25)
Response quality	Weak / incoherent	High on known questions	High and grounded
Hallucination risk	High (random)	Medium–high (unseen)	Low (grounded)
Robustness	Low	Medium	High
Complexity	Medium	Medium	High
Final suitability	Educational baseline	Intermediate solution	Recommended final choice

E. Final Choice

Based on this comparative analysis, the RAG approach was selected as the final architecture for the software-engineering chatbot. Three main reasons motivate this choice. First, RAG provides the highest robustness, since it can handle paraphrased and reformulated questions thanks to the combination of dense and sparse retrieval. Second, RAG offers the best contextualization, because the model generates an answer based on actual retrieved knowledge rather than relying solely on memorization. Third, RAG significantly reduces hallucinations by clearly separating the knowledge base from the generative model, which also makes the system easier to maintain and update over time.

V. LIMITATIONS AND FUTURE WORK

A. Limitations

The work presented in this paper has several limitations. First, the from-scratch approach is fundamentally constrained by the size and diversity of the available corpus; obtaining competitive results would require orders of magnitude more data and substantially more compute, which is not realistic in an academic project. Second, the fine-tuning approach is highly dependent on the quality and coverage of the dataset: the perfect validation scores most likely reflect an overlap between validation and training/indexed data rather than strong generalization to unseen questions or to other phrasings of the same question. Third, the RAG approach depends on the quality of the retrieval pipeline; if the knowledge base does not contain information relevant to the user query, the system may still produce a fluent but irrelevant answer. Fourth, the automatic metrics used (BLEU, ROUGE, semantic similarity) do not fully capture the pedagogical quality of a response, the clarity of explanations, or the appropriateness of the level of detail for a given learner. Finally, the chatbot remains limited

to the content of its knowledge base; questions outside the software-engineering domain or about very recent tools and frameworks may not be properly handled.

B. Future Work

Several directions can be explored to extend this work. The knowledge base could be enriched with additional software-engineering resources such as academic textbooks, course materials, and practical examples, in order to broaden coverage. A learned domain classifier could be added to detect out-of-domain questions and refuse to answer them gracefully, instead of relying on a simple confidence threshold. Human evaluation campaigns could be conducted with real students to measure the pedagogical usefulness of the answers, beyond purely automatic metrics. A conversational memory mechanism could be integrated to support multi-turn dialogues and follow-up questions. The Gradio interface could be replaced or extended with a richer web application supporting authentication, conversation history, and explicit user feedback. Larger or more recent LLMs (e.g., FLAN-T5-large, Mistral, Llama variants) could be tested with appropriate adaptation to the available hardware, possibly using more advanced PEFT techniques. Finally, the chatbot could be deployed in a real academic environment to gather usage data and iteratively improve both the knowledge base and the retrieval pipeline.

VI. Conclusion

This paper presented the development of a chatbot specialized in software engineering and a systematic comparison of three deep-learning-based approaches: a from-scratch Sequence-to-Sequence model with attention, a Fine-Tuned FLAN-T5 model coupled with semantic retrieval, and a complete Retrieval-Augmented Generation pipeline combining FAISS, BM25, a CrossEncoder reranker, and a LoRA-adapted FLAN-T5 generator. The three approaches were trained and evaluated on dedicated software-engineering corpora using standard metrics such as BLEU, Accuracy@K, MRR, ROUGE-1, ROUGE-L, and semantic similarity.

The experimental results showed clear differences between the three approaches. The from-scratch model, with a BLEU of only 0.0003, confirmed that training a generative chatbot from zero with a moderately sized corpus is not viable. The fine-tuning approach significantly improved response quality on known questions but remained sensitive to dataset coverage and to the risk of hallucination. The RAG approach reached a ROUGE-L of 0.6696, a ROUGE-1 of 0.6864, and a semantic similarity of 0.9449, while providing more robust, contextualized, and trustworthy answers. Based on these results, the RAG architecture was selected as the most suitable solution for the final chatbot.

From an academic perspective, this work illustrates how a clear methodological progression — from a from-scratch baseline to a fine-tuned pre-trained model and finally to a retrieval-augmented system — can be used to design and validate a specialized conversational assistant. From a practical perspective, the resulting chatbot, deployed via a Gradio interface with bilingual support, provides a concrete tool for students learning software engineering. We hope that the architecture and experimental insights presented here will serve as a useful starting point for future academic projects on domain-specialized chatbots.

Acknowledgment

The authors would like to thank the National School of Applied Sciences of Al Hoceima and their supervisor, Pr. Mohammed Cherradi, for their guidance and support throughout this project.

References

[1] A. Vaswani *et al.*, "Attention Is All You Need," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.

[2] C. Raffel *et al.*, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *J. Mach. Learn. Res.*, 2020.

[3] H. W. Chung *et al.*, "Scaling Instruction-Finetuned Language Models (FLAN-T5)," arXiv:2210.11416, 2022.

[4] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.

[5] H. Husain *et al.*, "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search," arXiv:1909.09436, 2019.

[6] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2014.

[7] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in *Int. Conf. Learn. Represent. (ICLR)*, 2015.

[8] J. Devlin *et al.*, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *NAACL-HLT*, 2019.

[9] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," arXiv:2106.09685, 2021.

[10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *EMNLP*, 2019.

[11] L. Wang *et al.*, "Multilingual E5 Text Embeddings: A Technical Report," arXiv:2402.05672, 2024.

[12] J. Johnson, M. Douze, and H. Jégou, "Billion-scale Similarity Search with GPUs," *IEEE Trans. Big Data*, 2019.

[13] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Found. Trends Inf. Retr.*, 2009.

[14] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global Vectors for Word Representation," in *EMNLP*, 2014.

[15] T. Mikolov *et al.*, "Efficient Estimation of Word Representations in Vector Space," arXiv:1301.3781, 2013.